Project Structure

> What each directory and key file is responsible for. Paths are repo-relative, commit `8de48cf`. Generated/vendored directories (`node_modules`, `dist`, `playwright-report`, `test-results`, `apps/api/storage`) are not described.

1. Top level

```
Aajiveka-Aplication/
├── apps/
│   ├── api/                NestJS 11 API (Prisma 7, PostgreSQL 16, Redis/BullMQ)
│   └── web/                React 19 SPA (Vite 7, Tailwind v4, TanStack Query, Zustand)
├── db/                     Restored legacy SQL Server database artefacts (reference only)
├── deploy/                 nginx config + deployment guide
├── tests/                  Real-API end-to-end suites + BillDesk payment-gateway stub
├── reference/              Extracted legacy ASP.NET sources (git-ignored)
├── scripts/                (empty)
├── .github/workflows/ci.yml CI: static checks, unit tests, Playwright, real-DB e2e, docker build, deploy
├── docker-compose.yml      Base stack: postgres, redis, api, web (nginx on :8080)
├── docker-compose.dev.yml  Publishes postgres :15432 and redis :16379 for host-run development
├── docker-compose.prod.yml Production overlay: RDS instead of postgres, S3 storage, SNS SMS, SMTP/SES email
├── .env.example            Names of the env vars compose needs (copy to .env; .env is git-ignored)
├── package.json            npm workspaces (apps/web, apps/api); scripts dev / build / lint / typecheck
├── README.md, CONTRIBUTING.md
└── docs/                   This documentation set
```

Untracked files present in the working tree: `Aajivika Book.pdf` (company brochure, untracked), `dump.rdb` (local Redis snapshot, git-ignored), `.env` (git-ignored, not read for this analysis).

2. `apps/api` — backend

```
apps/api/
├── Dockerfile, docker-entrypoint.sh   Build + run; entrypoint runs `prisma migrate deploy` then `node dist/main.js`
├── prisma.config.ts                   Prisma 7 config (DATABASE_URL, optional SHADOW_DATABASE_URL)
├── prisma/
│   ├── schema.prisma                  94 models + PaymentStatus enum, legacy table names via @@map
│   ├── migrations/                    34 migrations (Jul–Oct 2026)
│   ├── seed.ts                        Masters, plans, demo users (SEED_DEMO_USERS=1)
│   ├── migrate-data.ts                One-off legacy SQL Server → Postgres copy (destructive)
│   ├── data/                          India education / institute master data
│   └── tools/build-education-sql.ts   Generator for education master SQL
├── scripts/
│   ├── create-admin.ts                Bootstrap the first admin (prints a set-password link)
│   ├── publish-site-document.ts       Publish a downloadable site document (tblSiteDocument)
│   ├── seed-demo-applicants.ts        Demo data helpers
│   └── seed-applicant-profile-resume.ts
└── src/
    ├── main.ts                        Bootstrap: /api prefix, helmet, CORS, ValidationPipe, Swagger /api/docs, BigInt JSON
    ├── app.module.ts                  Module wiring + global guards (Throttler, JwtAuth, Roles) + request logger
    ├── config/env.ts                  Zod-validated environment (fails fast)
    ├── prisma/                        PrismaService / client factory (@prisma/adapter-pg), global module
    ├── common/
    │   ├── guards/                    jwt-auth.guard.ts, roles.guard.ts
    │   ├── decorators/                @Public, @Roles, @CurrentUser
    │   ├── filters/                   AllExceptionsFilter (error shape)
    │   ├── middleware/                RequestLoggerMiddleware
    │   └── email/                     EmailService + HTML templates (OTP, welcome, reset, status, interview, Q3 notice, job alert,
receipt)
    ├── shared/
    │   ├── roles.ts                   Role ids 1–7
    │   ├── status.ts                  JobMapStatus 1–23, job status 1–4, SubscriberStatus
    │   ├── hiring.ts                  Q3 vocabulary, IST date maths, stage derivation (stageOf)
    │   ├── matching.ts                Q2 criteria, weights, weightedScore, relevance threshold
    │   ├── screening.ts               Q1 statuses, priority, checklist
    │   └── pdf.ts                     PDF page counter
    └── modules/
        ├── auth/                      Login, refresh, OTP registration, employer registration, password reset, CSRF token, Redis
provider
        ├── candidates/                Candidate profile, sections, documents, saved jobs, alerts, prefs, referrals, privacy
        ├── files/                     Upload/download endpoints (resume, avatar, generic)
        ├── storage/                   StorageService + local / S3 drivers
        ├── jobs/                      Public search, job detail, apply; JobApplicationsService (apply + transitionStatus)
        ├── employers/                 Employer portal API (`/clients`): company, jobs, applicants, rounds, docs, analytics, billing
        ├── clients/                   Deprecated re-exports of employers (not imported)
        ├── companies/                 Public company search/page
        ├── recruitment/               Legacy QC pipeline + shared hiring services: referrals, interview rounds, offers, scoring,
mapping access, CV-expiry processor
        ├── q1/                        Q1 screening workspace + analytics
        ├── q2/                        Q2 matching workspace + analytics
        ├── q3/                        Q3 hiring-ops workspace: read models, referrals, interviews, documents, mail
        ├── admin/                     Admin panel API + staff account creation
        ├── payments/                  Razorpay (active), BillDesk (legacy), plans, subscriptions
        ├── exports/                   CSV / XLSX / PDF exports
        ├── notifications/             BullMQ queue + worker, SMS/email providers, SSE in-app notifications
        ├── audit/                     AuditService (tblAuditLog, tblSecUserLogin)
        ├── site-documents/            Public downloadable documents
        └── health/                    /api/health
```

Convention: each module has `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/*.dto.ts` (class-validator), optional `*.test.ts` (node:test).

3. `apps/web` — frontend

```
apps/web/
├── Dockerfile                Build with VITE_USE_MOCKS=0, VITE_API_BASE_URL=/api; serve with nginx (deploy/nginx.conf)
├── vite.config.ts            Vite + PWA plugin (manifest, shortcuts) + dev proxy /api → :4000
├── vitest.config.ts          Unit test config (jsdom, src/test-setup.ts)
├── playwright.config.ts      Mocked-API browser tests (e2e/)
├── vercel.json               SPA rewrite config (alternative hosting)
├── public/                   favicon, manifest.json, offline.html, robots/sitemap, locales, images, templates, mockServiceWorker.js
├── e2e/                      43 Playwright specs + support/ (auth helpers, fixtures, mocks)
└── src/
    ├── main.tsx              Entry: optional MSW (VITE_USE_MOCKS=1), providers, router, PWA prompts, offline sync
    ├── app/providers.tsx     QueryClient, Toast, auth bootstrap
    ├── routes/
    │   ├── router.tsx        All routes, lazy-loaded; role groups via ProtectedRoute
    │   └── ProtectedRoute.tsx Auth / role / onboarding guard
    ├── types/roles.ts        Role ids, labels, ROLE_HOME
    ├── lib/                  axios (interceptors), tokenStorage, queryClient (+ query keys), env, offlineQueue, analytics, ga4,
webVitals, errorTracking, sanitize, i18n, theme
    ├── hooks/                useSSE, useRealtimeNotifications, useOfflineSync, form/UX hooks
    ├── components/
    │   ├── ui/               Design-system primitives (Button, Input, Select, Table, Modal, Toast, searchable selects…)
    │   ├── layout/           PublicLayout, DashboardLayout, Navbar, Sidebar, menu.ts (role menus), NotificationBell
    │   └── *.tsx             Cross-cutting: CommandPalette, Seo, ErrorBoundary, PWA prompts, OfflineBanner, etc.
    ├── features/             Feature-sliced UI (pages + components + api hooks per domain)
    │   ├── auth/             Login (role "portals"), register + OTP, employer register, forgot/reset, auth.store (Zustand)
    │   ├── home/             Home page + /brochure (page-for-page brochure rebuild)
    │   ├── public/           About, Contact, Pricing, Blogs, Help, Legal, Career, Student guide, Salary insights, Company reviews…
    │   ├── jobs/             JobsLayout, search, detail, apply, search bar
    │   ├── candidates/       Candidate portal (portal/: layout, profile, wizard, tracker, interviews, documents, refer, account,
prefs, resume builder) + legacy pages (many unrouted)
    │   ├── payments/         Pricing plans, Razorpay checkout, membership, payment return
    │   ├── clients/          Public company page; deprecated client API + orphan components
    │   ├── recruitment/      Legacy QC / Q3 screens (/recruitment/*)
    │   ├── q1/  q2/  q3/     Staff workspaces (own shells, api hooks, components, formatters)
    │   ├── admin/            Admin panel pages + AdminShell + primitives
    │   ├── notifications/, messaging/, dashboard/, misc/, dev/   Mostly unrouted or placeholder (NotFound lives in misc/)
    ├── employer/             Employer portal: routes/, layouts/ (EmployerLayout), pages/ (Dashboard, ManageJobs, Applicants,
CompareCandidates, CompleteProfile, Analytics, Billing, Messages*, Notifications*, Settings*), services/, hooks/, components/
(*placeholders)
    ├── mocks/                MSW handlers + fixture data (dev/CI only)
    ├── styles/               Tailwind entry + animations
    ├── test/, test-setup.ts, test-utils/   Test setup and helpers
```

4. `db/` — legacy database reference (read-only)

Path	Content
`db/schema.sql`, `db/types.sql`, `db/indexes.sql`	Restored SQL Server DDL: 73 tables, 10 table-valued types
`db/procs/`	97 stored procedures (the legacy business logic the rebuild was derived from)
`db/procs_damaged_strings_extraction/`	Procedures recovered by string extraction
`db/seed/*.psv`	Master/lookup data (statuses, roles, cities, skills, documents, …) loaded by `seed.ts`
`db/ER-DIAGRAM.md`, `db/SCHEMA_REPORT.md`	Legacy model and inventory (partly stale; see DATABASE_SCHEMA)
`db/foreign-keys.psv`	Derived relations with confidence levels (used by `migrate-data.ts`)
`db/dead-tables.txt`	13 legacy tables omitted from the new schema
`db/columns.psv`, `db/tools/`	Column inventory and the Prisma-schema generator

5. `deploy/` and `tests/`

Path	Purpose
`deploy/nginx.conf`	Web container: serves the SPA, proxies `/api/` → `api:4000`, caching and security headers
`deploy/DEPLOYMENT.md`	Secrets, storage, payments, legacy migration, operations

Path	Purpose
`tests/e2e.mjs`	Real-API crawl of every public page and each role's pages
`tests/admin-e2e.mjs`	Real-API admin panel suite incl. responsive checks
`tests/billdesk-stub.mjs`	Local BillDesk gateway stand-in (`npm run dev:billdesk`)

6. Where to start reading

Goal	Start at
Understand roles and access	`apps/api/src/shared/roles.ts` , `apps/api/src/common/guards/*` , `apps/web/src/routes/router.tsx`
Understand the hiring pipeline	`apps/api/src/shared/status.ts` , `apps/api/src/modules/jobs/job-application.service.ts` (`transitionStatus`), `modules/q2` , `modules/q3` , `modules/recruitment/cv-referral.service.ts`
Understand data	`apps/api/prisma/schema.prisma` , DATABASE_SCHEMA.md
Run it	`README.md` , `CONTRIBUTING.md` , `deploy/DEPLOYMENT.md`